Introduction and the Machine Learning Workflow

Lesson 1 — Technologies for Artificial Intelligence

Fabio Antonini — Università degli Studi dell'Aquila

25 September 2026 · reading time about 70 minutes

Contents

Lesson plan

Time	Segment	Material
0:00–0:15	Course introduction and assessment	Slides 1–8
0:15–0:25	Environment check	`Course/Setup/`
0:25–0:50	What learning from data means	Slides 9–20
0:50–1:05	A short history	Slides 21–30, `Resources/`
1:05–1:15	**Break**	
1:15–1:45	The three kinds of learning	Slides 31–37, notebook 02

Time	Segment	Material
1:45–2:30	The end-to-end workflow, live	Slides 38–50, notebook 01
2:30–2:55	How models mislead	Slides 51–61, notebook 03
2:55–3:00	Homework set, questions	Slides 62–63
	Total	**180 minutes**

---

1. What this course is, and what it is not

This is a course about the **foundations** of machine learning: the methods, the mathematics beneath them, and the experimental discipline needed to tell a result that holds from one that merely looks good.

It is deliberately not a tour of current AI products. Large language models, transformers and agent architectures are covered elsewhere in the programme. What you learn here is what those systems are built on, and — more importantly — how to judge whether any such system actually works.

You arrive with an advantage and a gap, and it is worth naming both.

The advantage is that you can already program. You will not spend this course fighting Python, and you will be able to implement methods from scratch rather than only calling them. That matters: a method you have built once is a method you understand.

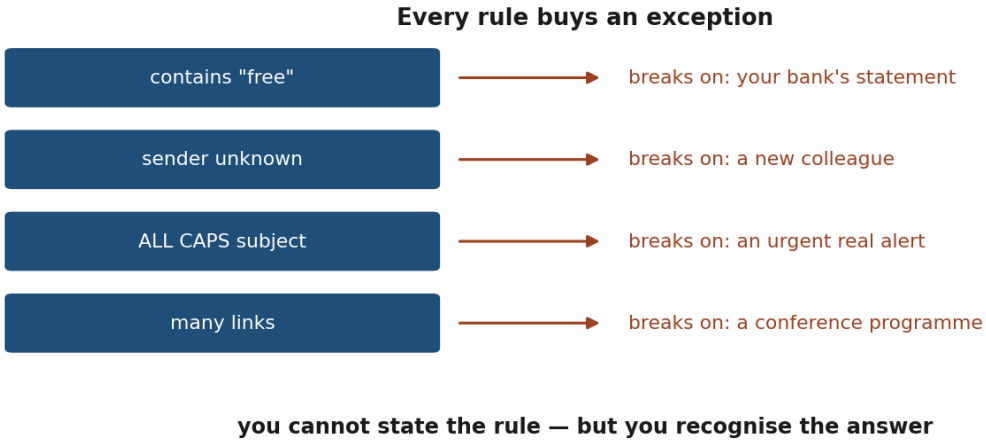
The gap is that programming ability lets you produce results faster than you can judge them. Within a few weeks you will be able to write thirty lines that yield 95% accuracy on something. The hard question — the one this course exists to answer — is whether that number means anything at all. Lesson 5 is devoted to it, and Lesson 1 begins raising it.

---

2. What learning from data means

Start with the problem that machine learning solves, and notice that it is a problem about *specification*, not about computation.

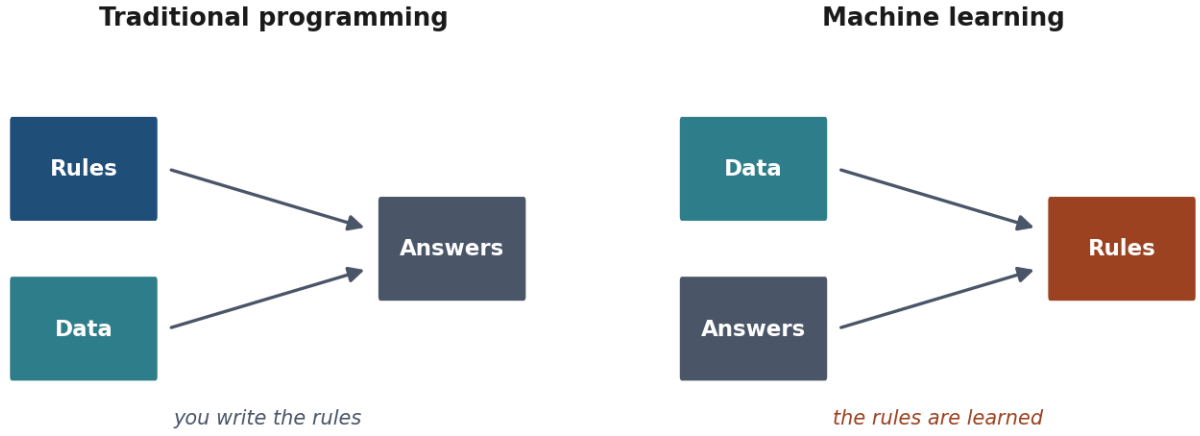
Suppose you must write a program that decides whether an email is spam. You could try to write the rules by hand: flag messages containing certain words, or arriving from certain domains. You would produce something that works for a week. Spam adapts; your rules do not; the list of exceptions grows until nobody can maintain it.



The problem you cannot specify. Every rule catches some spam and some legitimate mail, and the list never converges.

The difficulty is not that the program is hard to write. It is that **you cannot state the rule**. You recognise spam instantly and cannot articulate how.

Machine learning inverts the approach. Rather than specifying the rule, you supply examples and let a procedure search for a rule consistent with them.

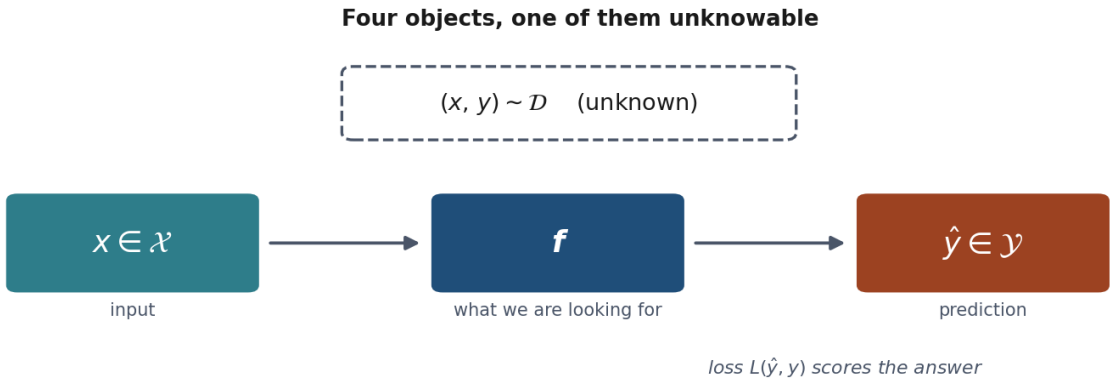


The inversion. Programming takes rules and data and produces answers; learning takes data and answers and produces the rules.

2.1 The formalisation

This much can be made precise, and doing so pays off later.

We have an input space $\mathcal{X}$ (emails, tumour measurements, images) and an output space $\mathcal{Y}$ (spam or not, malignant or benign, a price). We assume pairs (x, y) are drawn from some fixed but unknown probability distribution $\mathcal{D}$ over $\mathcal{X} \times \mathcal{Y}$.



The setup in one picture: an unknown distribution, a sample drawn from it, a model chosen using that sample, and the world it will actually meet.

We want a function $f : \mathcal{X} \to \mathcal{Y}$ that predicts well. "Well" needs a definition, so we introduce a **loss function** $L(\hat{y}, y)$ measuring the cost of answering $\hat{y}$ when the truth is y. The quantity we actually care about is the **expected risk**:

$$R(f) = \mathbb{E}_{(x,y)\sim\mathcal{D}}\left[L(f(x), y)\right]$$

This is the average loss over all data the world might produce — including data that does not exist yet. It is the thing we want to make small.

And it is **not computable**, because $\mathcal{D}$ is unknown. All we hold is a finite sample $S = \{(x_1, y_1), \ldots, (x_m, y_m)\}$, so we compute the **empirical risk** instead:

$$\hat{R}_S(f) = \frac{1}{m}\sum_{i=1}^{m} L(f(x_i), y_i)$$

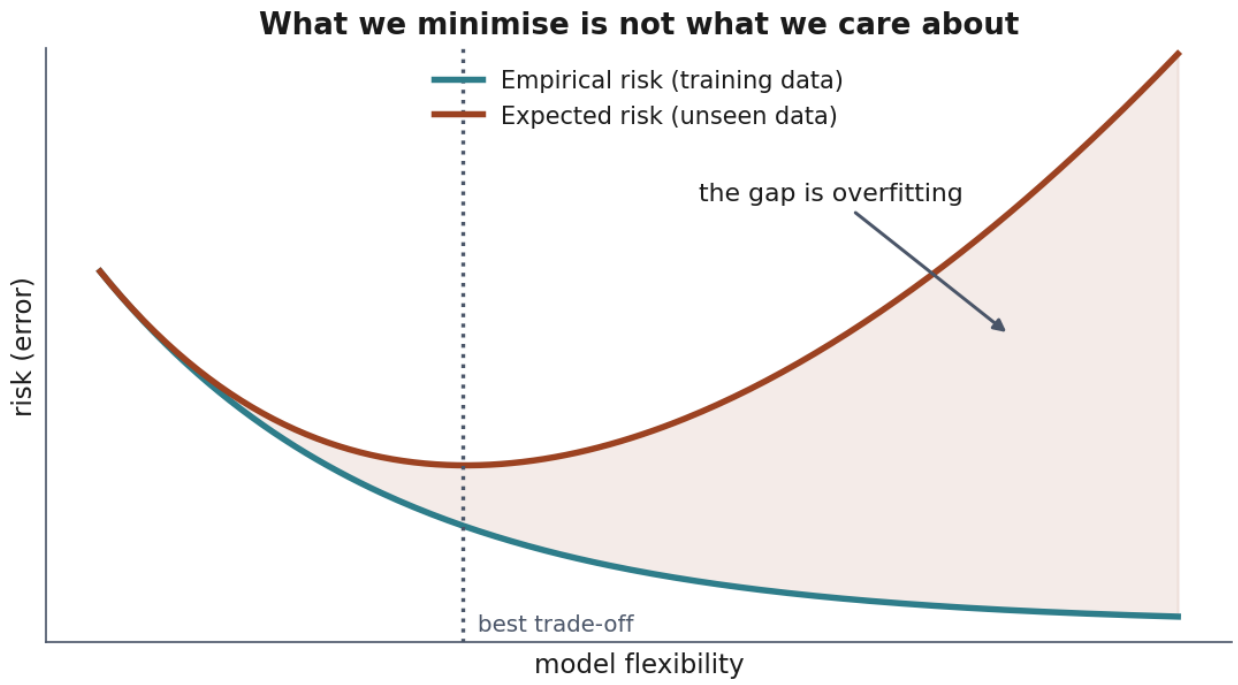
and choose the f that makes it small. This is *empirical risk minimisation*, and essentially every method in this course is an instance of it.

What L actually is. So far it is a placeholder, and it should not stay one. The simplest choice for a classifier is the **zero-one loss**: $L(\hat{y}, y) = 0$ when $\hat{y} = y$, and 1 otherwise. Under it the expected risk is the probability of being wrong and the empirical risk is the error rate on the sample — so **accuracy is** $1 - \hat{R}_S(f)$, and every score the notebooks print is an empirical risk wearing a friendlier name. Notebook 01 reaches 0.986 on its 143 test examples: two mistakes, so $\hat{R}_T(f) = 2/143 = 0.014$.

And the choice is not cosmetic. L is where you state what counts as a bad mistake, before any model exists. Zero-one loss says every error costs the same, which is almost never true — a missed malignant tumour and a false alarm are not the same event, and Section 3 prices exactly that. Lesson 3 replaces zero-one with squared error for continuous targets, Lesson 4 with cross-entropy, and Lesson 4 goes further and gives the two kinds of mistake separate prices. Changing L changes which f wins, which is why it is a modelling decision and not a technicality.

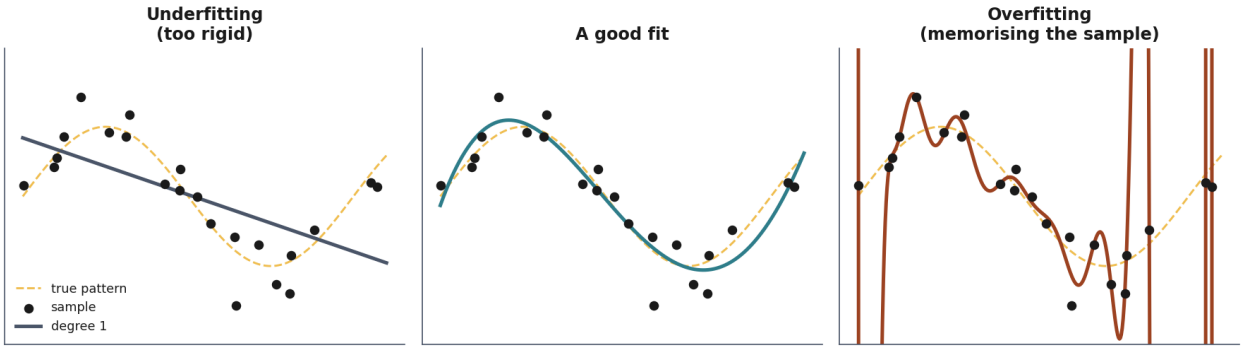
2.2 The gap that explains the whole course

Here is the crux. We minimise $\hat{R}_S(f)$ but care about $R(f)$, and the two are not the same number.



The quantity we want and the quantity we can compute are not the same. Everything in this course is about keeping the gap between them small and honest.

A sufficiently flexible f can drive the empirical risk to zero by memorising the sample — storing every pair and reciting the answer. Its expected risk would be terrible: it has learnt the sample rather than the pattern. That is **overfitting**, and it is the central difficulty of the field.



The same twenty-two points fitted three times. The dashed line is the pattern the data really came from, which no method gets to see. Left, a straight line is too rigid to follow it. Right, a degree-18 polynomial passes almost exactly through every sample point and shoots off the scale between them — its empirical risk is nearly zero and it has learnt the noise, not the pattern. The middle one is what we want, and nothing in the training error distinguishes it from the right-hand one.

So the practical question is never "how well does the model do on the data I fitted it to?" — that number is always optimistic and sometimes meaningless. It is "how well will it do on data it has never seen?"

Since $R(f)$ cannot be computed, we estimate it: hold out part of the sample, never let the fitting procedure touch it, and measure there. **That is the entire reason the test set exists.**

2.3 Why holding data out actually works

The picture first. Imagine a revision session the week before an exam, in which the lecturer works through exactly the exercises that will be on the paper — not deliberately, simply because those are the exercises on their mind. The class does very well. Nobody cheated, and every mark was earned. But the marks now measure how well those particular exercises were rehearsed rather than how well the subject is understood, and nothing in the marks themselves tells the two apart.

A test set is that exam paper. It measures generalisation only for as long as nothing has rehearsed on it — for as long as nothing about it influenced the model. That is the entire content of the rule, and the argument below is that sentence made precise.

The rule is easy to state and easy to treat as hygiene. It is worth seeing why it is not — the justification is short, and it tells you exactly when the rule has been broken.

The difficulty is not that a sample is small or unrepresentative. It is this:

> If the same data both **chooses** the model and **judges** it, the judgement is no longer an unbiased estimate. It is systematically optimistic.

An analogy that lands in a lecture: the exercises rehearsed in the revision session cannot also be the exercises that are marked. Nobody cheated — the same questions simply did two jobs at once, and the mark inherited the optimism.

Now the reason a held-out set repairs this. Suppose we fix a function f using the training data alone, and then evaluate it on a test set T drawn from the same distribution $\mathcal{D}$ and never consulted while choosing f. Because T is independent of f, each test example is an unbiased draw of the loss, and so

$$\mathbb{E}_{T\sim\mathcal{D}^m}\left[\hat{R}_T(f)\right] = R(f)$$

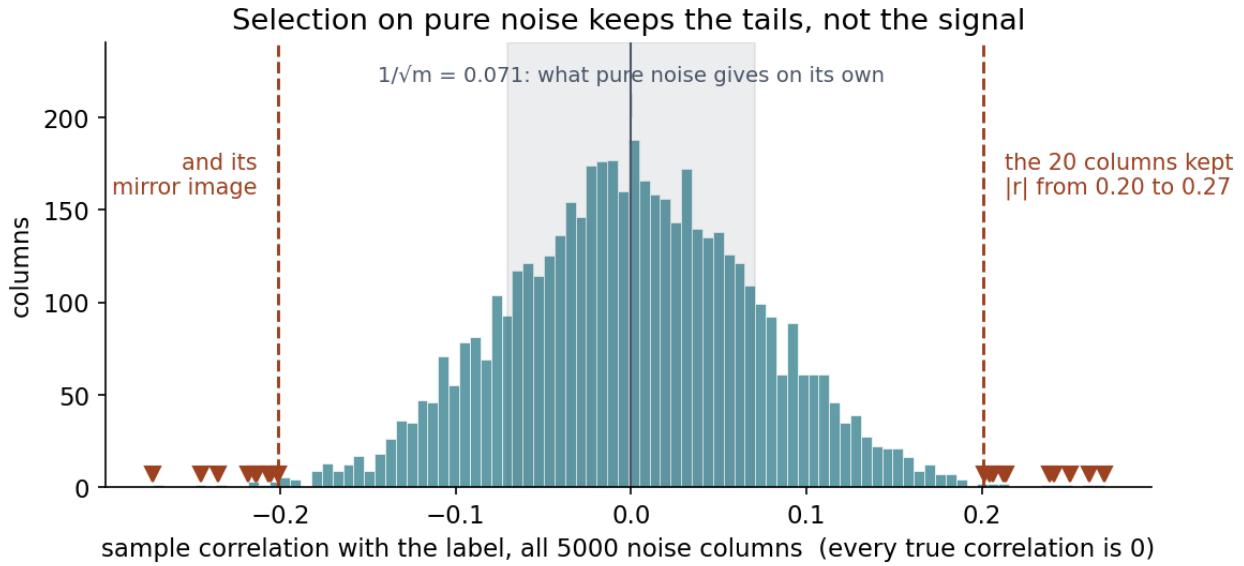
The empirical risk on the test set is an **unbiased estimator of the expected risk** — the quantity we said was not computable. That equation is what the test set buys, and it is worth writing on the board.

Read the expectation carefully, because it is easy to over-read. The subscript $T \sim \mathcal{D}^m$ says the average is taken over *all* the test sets of size m that could have been drawn from $\mathcal{D}$ — not over the single test set you actually hold. Your test set is a draw from that collection, and it is still too optimistic or too pessimistic by some amount. What the equation promises is only that the amount is not *systematically* in one direction: repeat the whole experiment many times with fresh test sets and the scores centre on $R(f)$.

So "unbiased" is a statement about the *procedure*, not a guarantee about *your number*. It says the target is the right one; it says nothing about how far a single shot lands from it. That second question — the spread rather than the centre — is what Section 2.4 measures, and it is why a test score quoted without an error bar is incomplete.

Notice precisely what the argument depends on: **the independence of T from f**. Not on the size of the split, not on the proportion, not on stratification. The moment the test rows influence any decision — a mean used for scaling, a ranking used to select features, a comparison used to pick a model — f ceases to be independent of T, the equality above fails, and the estimate becomes optimistic by an unknown amount.

That is why the rule is "nothing is learned from the data before the split", not "remember to keep some data aside". Notebook 03 breaks exactly this independence and obtains **77% accuracy on labels generated by a coin flip**.



How that happens, on the very data that produces the 77%. Every one of the 5000 columns has a true correlation of zero with the label; a sample correlation measured on 200 rows lands within about $1/\sqrt{m} = 0.07$ *of zero, which is the shaded band. Look at where the triangles are: the twenty columns the selector kept are the extremes of that distribution, in both tails, with no signal in any of them. And those extremes were read off all 200 rows — including the 60 that were about to become the test set.*

Why the selector lands there is worth stating, because it is not bad luck. `SelectKBest` is a ranking, not a test: it never asks "does this column carry signal?", a question it could answer *no* to. It asks "which are the best k?", sorts all 5000 by strength of association and returns the top of the list. It cannot return nothing. So when every true correlation is zero, sorting by strength is sorting by luck, and "the top 20 out of 5000" is a description of the tail. Selecting the extremes is not an accident of the method — it is exactly what the method was asked to do.

The size of those extremes follows from the same two numbers as everything else here. The largest of n draws with spread σ sits near $\sigma\sqrt{2\ln n}$, so with $\sigma = 1/\sqrt{200} = 0.071$ and $n = 5000$ the biggest is predicted at $0.071 \times 4.13 = 0.29$. Notebook 03 measures 0.27, and the twenty kept columns run from 0.20 upwards — between **2.8 and 3.9 standard deviations from zero**. A correlation of 0.27 on 200 rows would look like a finding in a paper. It is the maximum of five thousand coin flips.

The test information did not enter through the rows. It entered through the choice of columns, which is why nothing in the code looks wrong. And note what is *not* at fault: Section 6's pipeline makes the identical `SelectKBest` call correctly, by placing it where it only ever sees training rows. The tool is fine; the line it was called on was not.

2.4 How much to hold out, and how much to trust it

The picture first. A test score is a measurement, and measurements have error bars. Ask a hundred people whether they will vote for a party and you would not report the result to three

decimal places; ask a thousand and the figure steadies. A test set works the same way: the fewer examples it holds, the noisier the number it gives you.

That is where the trade-off comes from. Move examples into the test set and the measurement steadies but the model has less to learn from; move them the other way and you get a better model whose quality you know less precisely.

Size is only half of it. Even at a fixed size, *which* rows land in the test set is a draw of its own, and Section 7 measures how much that alone moves the score.

Two practical consequences follow, and both surprise people.

The estimate has a variance of its own. An accuracy measured on m test examples is a proportion — it is the empirical risk under the zero-one loss of Section 2.1, counted over m independent draws — so its standard error is roughly

$$\mathrm{SE} \approx \sqrt{\frac{p(1-p)}{m}}$$

In Notebook 01 the test set holds 143 examples and the accuracy comes out at 0.986. That gives a standard error of about **one percentage point**, so reporting "0.986" to three decimal places claims a precision we do not have: the third digit is noise. This is the same observation that Section 7 makes about single splits, arriving from the other direction.

And the formula has a limit worth meeting here. It is a normal approximation, and a normal approximation needs enough events on both sides to be trusted — the usual rule of thumb asks for at least five. But 0.986 of 143 is **two errors**, and $m\,p(1-p) \approx 2$. Push the approximation anyway and it returns an interval running up to **100.5%**: it admits accuracies above 100%, which settles the matter without any further argument. A method that does not lean on the approximation puts the lower bound at **95.0%** instead — a point and a half below what the approximation promised, and in the flattering direction. Lesson 5 takes up how such an interval is properly built. What belongs here is the habit: ask whether a formula is admissible before quoting the number it gives you.

Hence the trade-off in choosing the split. A larger test set gives a more stable estimate — the standard error falls as $1/\sqrt{m}$. A larger training set gives a better model. With thousands of examples, holding out 20-30% costs little and buys a reliable number. With a few hundred, both sides hurt at once, which is precisely the situation cross-validation is designed for. Lesson 5 takes it up.

2.5 One simplification we are making today

This lesson speaks of a training set and a test set. In practice three roles are needed:

Set	Used for	How often you may look
Training	Fitting the model	Freely
Validation	Choosing between models and settings	Freely
Test	The final reported number	Once

Train and test today; lesson 5 adds the validation set that every choice should actually be made on.

Today we do not need the middle one, because we make no choices: one model, no tuning, no comparison. As soon as you compare ten candidates and report the best, you have *selected* using whatever data you compared them on — and if that was the test set, the winning number is optimistic again, for exactly the reason in Section 2.3. The selection is a decision, and decisions are what the test set must stay independent of.

Everything in Lesson 5 elaborates these three sections.

---

3. When not to use machine learning

An unfashionable section, and the most immediately useful one.

When the rule is known. If you can state the rule, write it. Nobody should train a classifier to detect whether a number is even. This sounds obvious and is violated constantly, usually because ML is the interesting option.

When you cannot obtain representative data. A model learns the distribution it was trained on. If your sample comes from a different population than the one you will deploy on — different hospital, different season, different user base — the model's measured performance says little about its real performance.

When errors are catastrophic and unexplainable. Learned models are wrong sometimes, in ways that are hard to predict and often hard to explain. If a mistake is unrecoverable, and no human reviews the output, the question is not whether the accuracy is high enough but whether anyone can tell when it has failed.

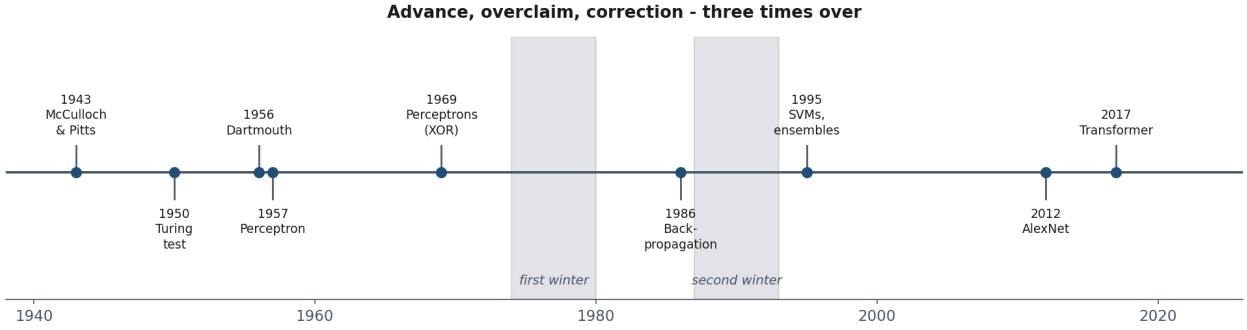
When the data encodes an injustice you would be automating. If historical decisions were biased, a model fitted to them reproduces the bias — with an appearance of objectivity that makes it harder to contest. The model is not neutral because it is mathematical; it is a compressed summary of the decisions in its training data.

When a simple baseline is already sufficient. Often a threshold on one variable solves 90% of the problem, is understandable by everyone, and needs no maintenance. Establishing baselines first (Section 6) is partly a way of discovering this before building something complicated.

---

4. A short history, and what it teaches

The field's history is worth an hour not for the anecdotes, but because its pattern repeats and you are living through another iteration of it.



Seventy years in one picture. The pattern worth recognising is the rhythm: a genuine advance, claims well beyond the evidence, and a correction expensive enough to cost a generation of funding.

1943 — the first artificial neuron. McCulloch and Pitts model a neuron as a threshold unit: it fires when the weighted sum of its inputs exceeds a threshold. The model is a claim that thought might be computation.

1950 — Turing's imitation game. Turing sidesteps "can machines think?" and replaces it with a question that can be tested by observation — an early instance of the move this course keeps making: replace an unanswerable question with a measurable proxy, and stay aware of the gap between them.

1956 — Dartmouth. The term *artificial intelligence* is coined at a summer workshop whose proposal suggests that significant progress could be made in two months by ten people. The optimism is not incidental; it is the pattern.

1957 — the perceptron. Rosenblatt builds a learning machine: weights adjusted from examples rather than set by hand. It is the direct ancestor of everything in Lessons 9 and 10. Press coverage promises machines that will walk, talk and be conscious.

1969 — the first winter begins. Minsky and Papert prove the perceptron cannot represent XOR. The limitation is real but narrower than the reception suggests — it applies to a single layer, and multi-layer networks were not yet trainable. Funding collapses for a decade.

1986 — backpropagation popularised. Rumelhart, Hinton and Williams give a practical way to train multi-layer networks, removing the objection. Enthusiasm returns; a second winter follows in the early 1990s when results again fall short of promises.

1990s–2000s — statistics takes over. Support vector machines, ensembles and probabilistic methods dominate. The framing shifts from "simulating intelligence" to "estimating functions from data" — a retreat in ambition and an advance in rigour. Most of this course lives here.

2012 — deep learning arrives. AlexNet wins ImageNet by a wide margin. The ideas are largely from the 1980s; what changed is data and compute. This is worth pausing on: the breakthrough came from scale, not from a new principle.

2017 onwards — scale. The transformer architecture, then models trained on internet-scale corpora. Capabilities that surprised most researchers, alongside claims that would have been familiar

to anyone reading press coverage of the perceptron.

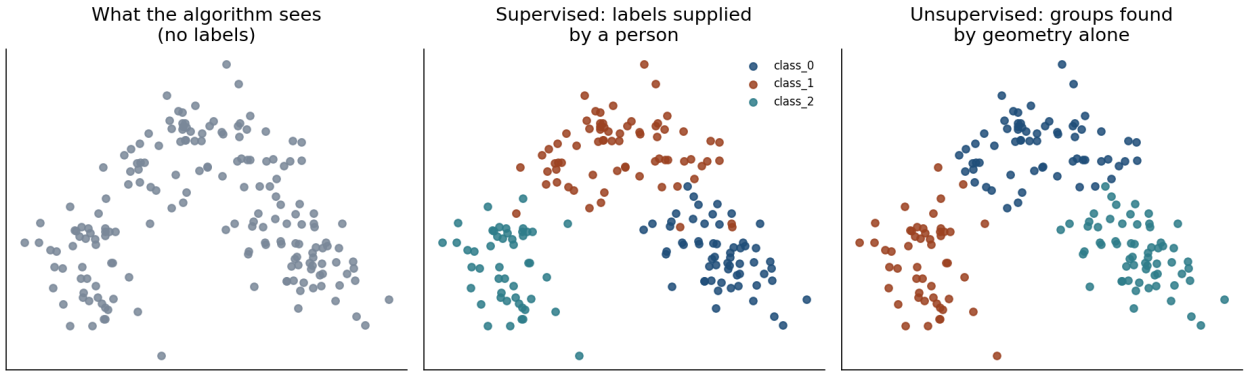
What the pattern teaches. Each cycle featured genuine advances, followed by claims outrunning evidence, followed by correction. The technical lesson is that progress has come as much from data and computation as from new ideas. The professional lesson is that the ability to distinguish a demonstrated result from an extrapolated promise is a durable skill — and it is, again, the skill this course is built to teach.

`Resources/history_of_ai.md` covers this in more depth, with the primary sources.

---

5. The three kinds of learning

The standard taxonomy divides methods by **where the target comes from**. That is the distinction that matters; the algorithms often overlap.



One dataset, three questions. What changes is not the data but what you ask of it.

Notebook 02 runs all three on one table, and it is the only place in this lesson where they can be compared without anything else changing: 178 Italian wines, 13 chemical measurements each, from three different cultivars. Every result quoted in this section comes from it, and the numbers are worth reading against each other rather than one at a time.

5.1 Supervised learning

Each example carries a label somebody produced: (x_i, y_i) pairs. The model learns the mapping and can be checked against ground truth.

When y is a finite set of categories the task is **classification**; when it is continuous, **regression**. Lessons 3, 4, 6 and 7 are all supervised.

The cost is the labels. In practice this — not algorithms, not compute — is what limits most projects.

Notebook 02 does both on the wine table. Predicting the cultivar — a category, so classification — reaches **0.981** accuracy. Swapping the target for colour intensity, a continuous quantity, makes the identical pipeline a regression, scored with R^2 at **0.622**. Nothing changed but which column was called the answer.

5.2 Unsupervised learning

No targets. The data is $\{x_1, \ldots, x_m\}$ and the goal is structure: groups (clustering), a lower-dimensional description (dimensionality reduction), or unusual points (anomaly detection). Lesson 8.

What is missing is not a loss. It is easy to read "no targets" as "nothing to minimise", and that is wrong. k-means minimises the total squared distance from each point to its nearest cluster centre — the **within-cluster sum of squares (WCSS)** — which is an empirical risk in exactly the sense of Section 2.1, with the loss measured against the model's own summary of the data rather than against a label. Lesson 8 derives it and proves the algorithm decreases it at every step. Principal component analysis (PCA), also Lesson 8, minimises reconstruction error the same way.

What is missing is **ground truth to check the answer against**. An algorithm will always return groups, whether or not the data contains any. Whether they correspond to anything you care about is a judgement you must make, and it cannot be delegated to a metric. That is the difficulty — and it is a problem of validation, not of optimisation.

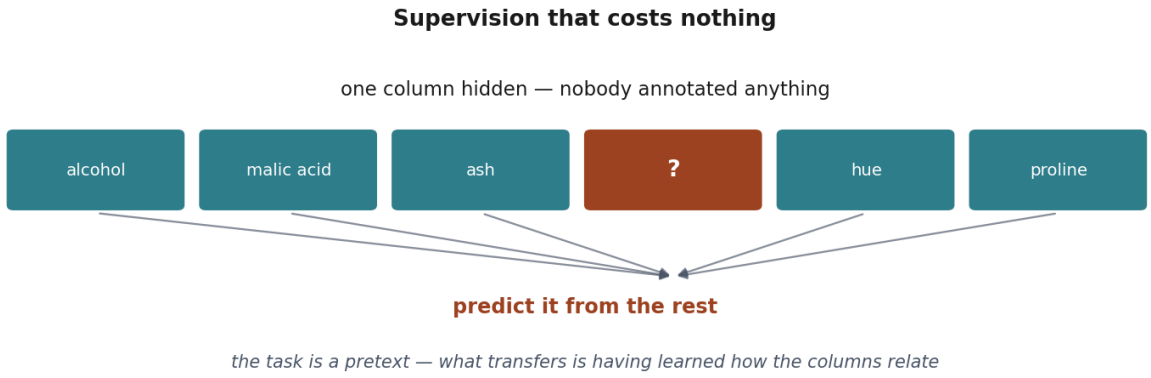
Notebook 02 makes that concrete by cheating deliberately. It throws the cultivar labels away, runs k-means, and then — only as a teaching check, never as something available in a real unsupervised problem — compares the groups it found against the labels it hid. They agree at **0.897**. That is a happy accident of this dataset, where the chemical groups genuinely are separated in space, and the notebook says so: do not expect it. In a real problem there is no such comparison to make, which is exactly the point.

The gap of Section 2.2 does not disappear along with the labels. WCSS falls as the number of clusters rises, and with one cluster per example it is exactly zero. That is the same memorisation that drives a flexible model's empirical risk to zero, in different clothes: nobody labelled anything, and it is still overfitting. Lesson 8 spends its first half on how to choose that number honestly.

5.3 Self-supervised learning

A target is manufactured from the input: hide part of each example and train the model to reconstruct it from the rest. Nobody annotates anything, yet the supervision is genuine.

Notebook 02 hides one measurement, `flavanoids`, and predicts it from the other twelve: $R^2 = 0.816$. Compare that cell with the regression in Section 5.1 and note that they are **mechanically identical** — the same estimator, the same call, the same kind of score. What differs is only where the target came from. A person recorded the cultivar; nobody produced the flavanoid column as a label, it was already in the table. That difference costs nothing and is the whole of the idea.



Where the labels come from when nobody labelled anything: hide part of the input and ask the model to reconstruct it.

The reconstruction task itself is of no interest — it is a *pretext*. The point is that solving it forces the model to represent how the parts of an input relate, and that representation transfers to tasks you do care about.

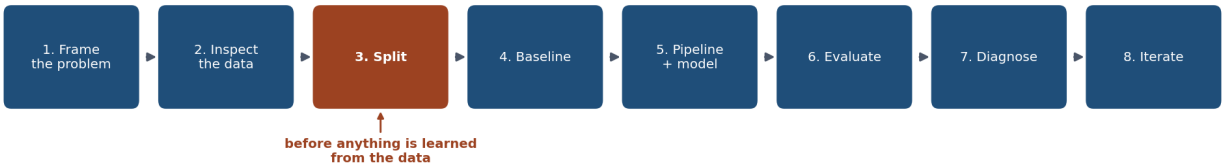
Formally nothing new is happening. The target y is manufactured from x, so everything in Section 2 applies unchanged — the same expected risk, the same empirical risk, the same gap between them, the same reason for holding data out. What changes is only that the labels are free.

This is how modern large models are trained, and it explains their scale: text and images exist in enormous quantities, and this technique needs no annotation. We do not pursue it further here, but you should recognise it.

---

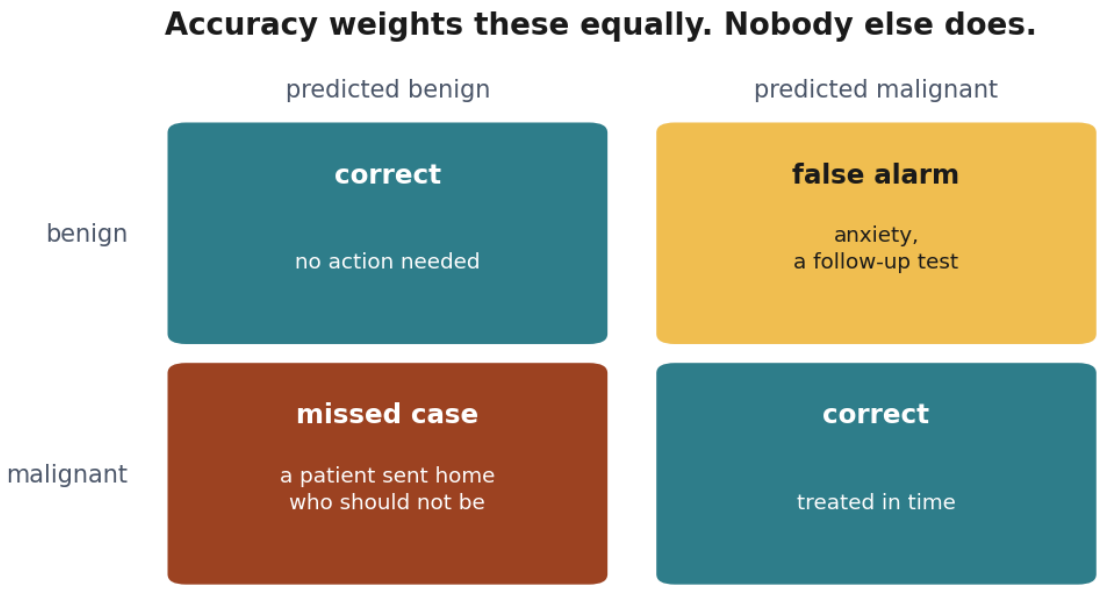
6. The end-to-end workflow

The order of these steps is not a convention. Several of them are only valid in this order.



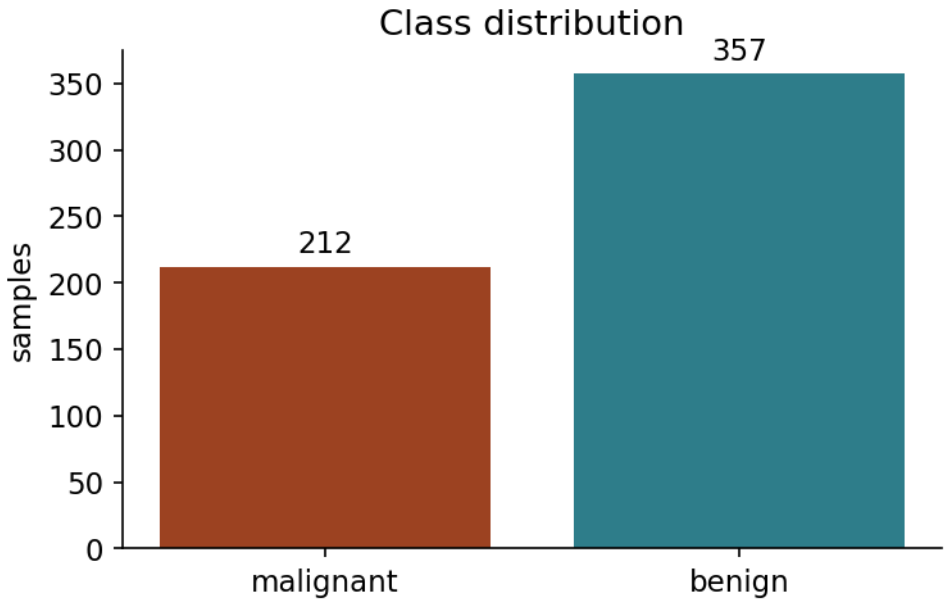
The cycle, and the order that matters most: the split comes before anything is fitted, not after.

1. Frame the problem. What is predicted, from what, and what would make an error harmful? Which mistake is worse? Answer before modelling — the answers determine which metric is honest, and a metric chosen after seeing results is a metric chosen to flatter them.



Two of these four cells are failures, and they are not the same size: a false alarm costs a follow-up test, a missed case sends a patient home who should not go. Accuracy adds all four up as though they weighed the same, which is why the metric cannot be chosen until this question has been answered.

2. Obtain and inspect the data. Size, types, missing values, class balance, obvious errors. Look before transforming.



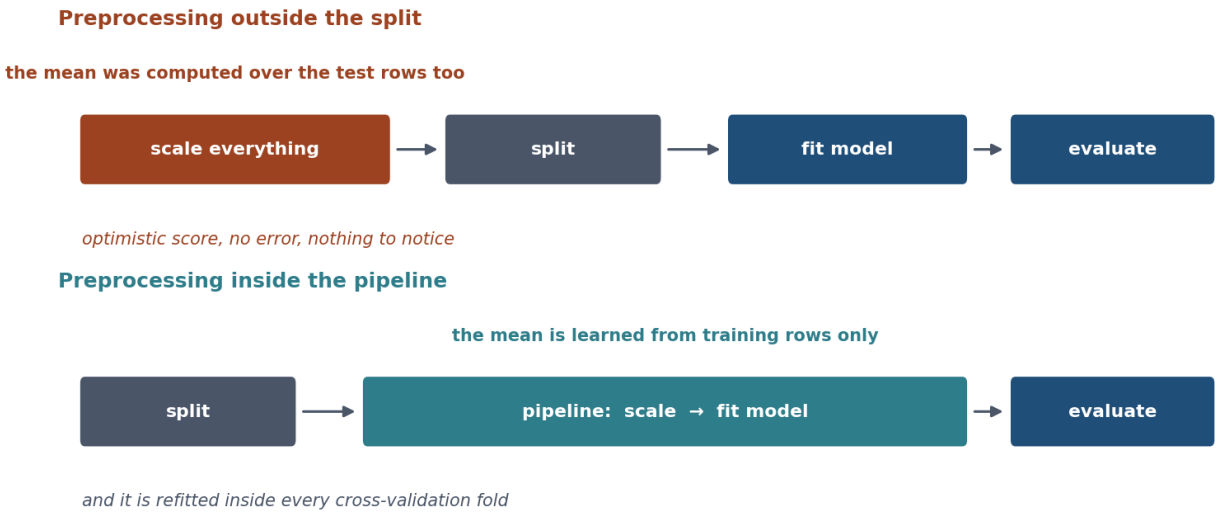
Look before you touch anything. The class balance decides which metrics will mean something later: 357 benign against 212 malignant, so answering "benign" every time is already right 62.7% of the time.

3. Split. Hold out a test set **now**, before anything is learnt. Every subsequent decision uses the

training portion only. See Sections 2.2-2.3 for why this is the load- bearing step.

4. Baseline. The simplest thing that could work: predict the majority class, predict the mean, threshold a single variable. Without this, a score has no meaning — you cannot tell 95% accuracy that is excellent from 95% that is worse than answering "no" every time.

5. Preprocess and model, inside a pipeline. Scaling, encoding and imputation all *learn* from data (a mean, a set of categories), so they must be fitted on training data only. A pipeline enforces this structurally. Doing it by hand works until the day it does not, and that day is silent.



The same two steps in the two possible orders. Look at where "split" sits on the top row: after the scaling, so the mean subtracted from the training rows was computed with the test rows included. Only the bottom row survives cross-validation honestly, and nothing about the top row raises an error.

6. Evaluate with a metric that matches the cost of being wrong. Accuracy weights every error equally. Real problems rarely do.

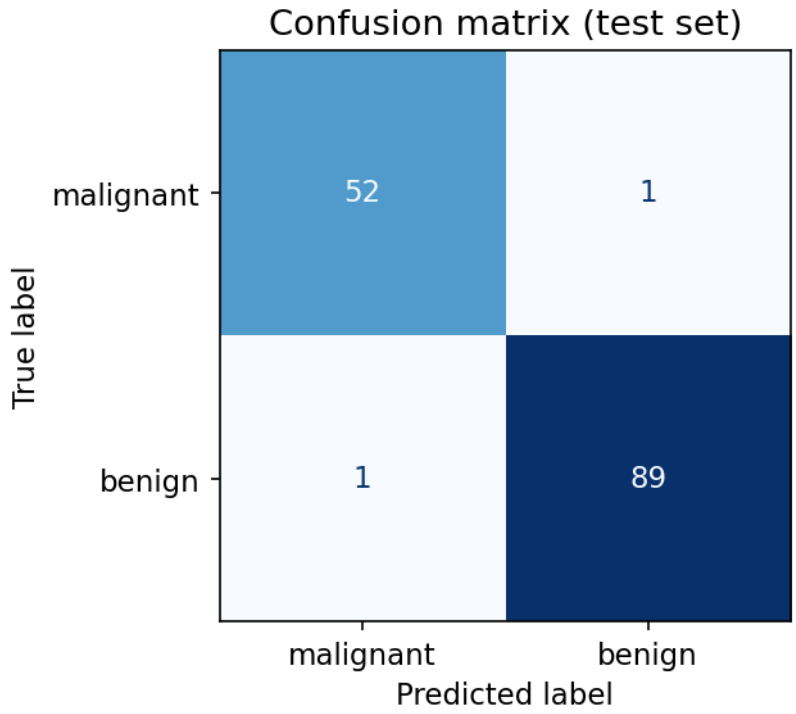
7. Diagnose. Where does it fail, and is the failure systematic? Error analysis tells you more than a decimal place of accuracy.

8. Iterate — with discipline. Each look at the test set to guide a decision spends some of its independence. Keep a validation set for choices, and reserve the test set for the end. Lesson 5 makes this precise.

---

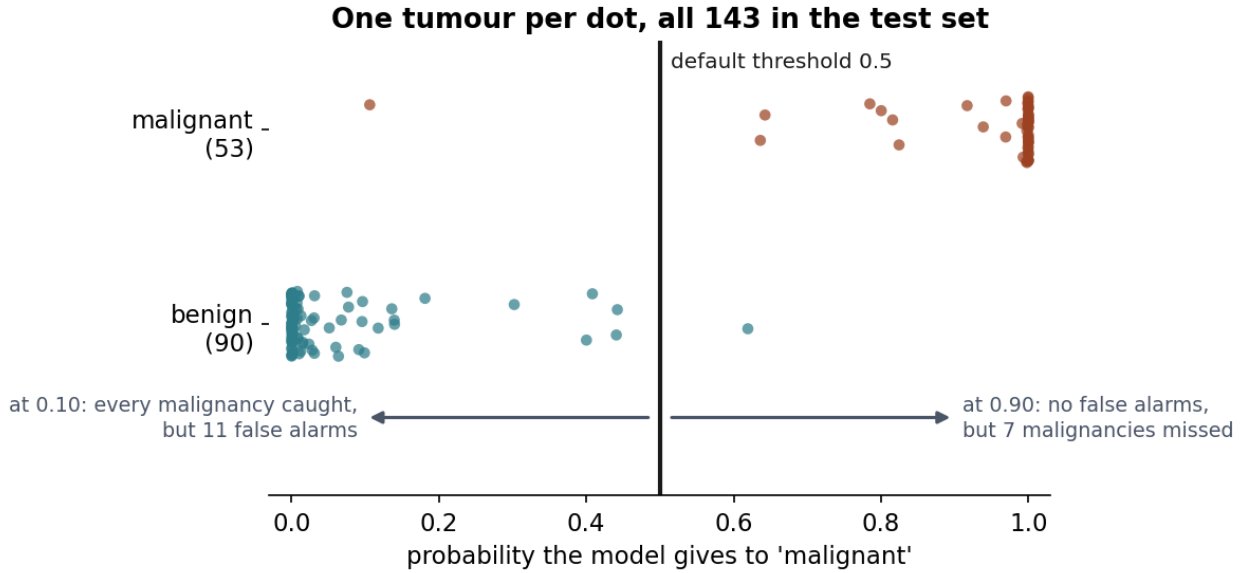
7. How models mislead

A reported score is a summary, and a summary is something with information removed. Before the four failure modes, it is worth seeing what Notebook 01's headline number left out: 0.986 is four separate counts added together, and only two of the four are successes.



What a single accuracy figure hides: four outcomes collapsed into one number. Lesson 4 takes this apart properly.

Those four counts are not a property of the model alone. They are what the model produces *once a threshold has been chosen*, and on this problem nobody chose it.



One dot per tumour, on two rows: the 53 malignant cases above, the 90 benign below. Left to right is the probability the model gave to "malignant", and the black line is the 0.5 default — everything to its right is called malignant, everything to its left benign.

Most of that picture is the model being certain, and it is the part to ignore: 105 of the 143 tumours

sit within 0.02 of one end or the other. That crowd is where 0.986 accuracy comes from, and it tells you nothing about the threshold, because no threshold you would consider moves any of it.

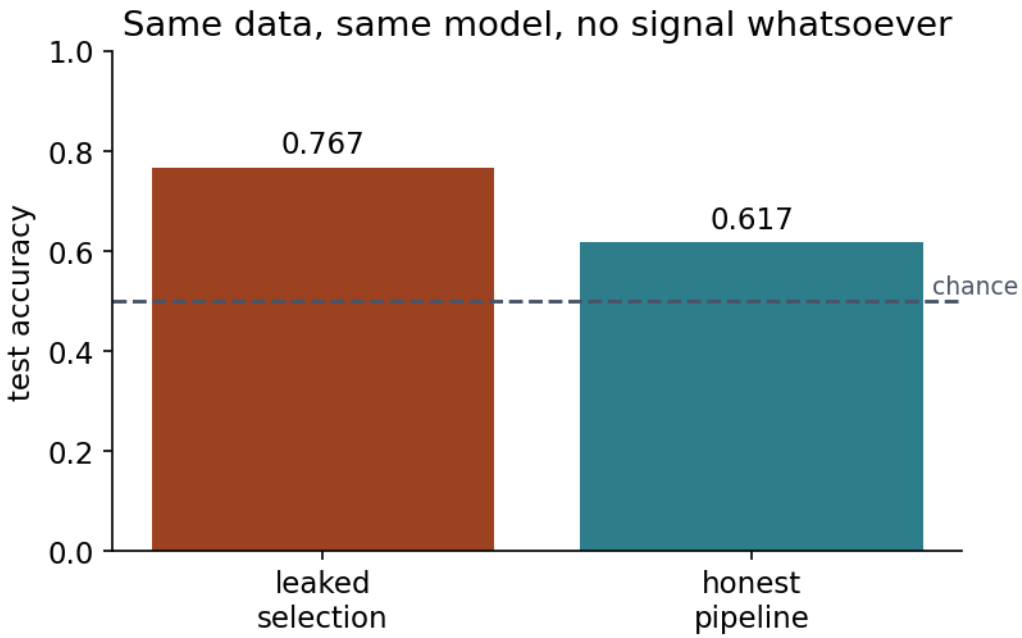
The lesson is in the middle. Between 0.11 and 0.62 lies everything the model was unsure about: **12 tumours, 1 malignant and 11 benign**. At the 0.5 default only two of those twelve fall on the wrong side — the malignancy at 0.11, missed, and the benign case at 0.62, a false alarm. That is the 1 and the 1 in the confusion matrix above, and it is the whole of the model's error.

Now slide the line, and watch the band rather than the numbers. Move it **down to 0.10** and it passes below the entire band: the missed malignancy is caught, and every one of the 11 uncertain benign tumours turns into a false alarm. Those 11 are not a figure quoted from a table — they are the dots you can count to the left of the line. Move it **up to 0.90** instead and no false alarm survives, at the price of 7 malignancies missed.

Nothing about the model changed between those two sentences. The same fitted coefficients, the same probabilities, the same 143 tumours; only the line moved. That is what it means to say the threshold is a decision rather than a calculation — and deciding it requires knowing what a missed cancer costs against what a needless biopsy costs, which is not a question the data can answer.

Four failure modes, all producing numbers that survive casual review, all demonstrated in notebook 03.

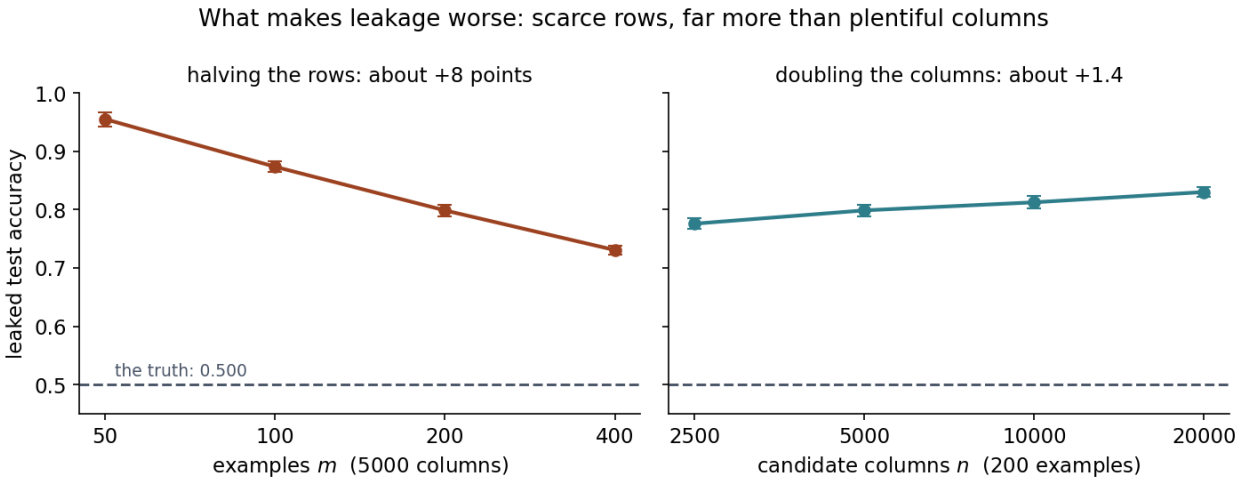
Data leakage. Information from the test set reaches the training procedure. Selecting features on the full dataset, scaling before splitting, or including a column recorded after the outcome. Leakage inflates scores without any error being visible in the code: notebook 03 obtains 77% accuracy on data that is pure random noise.



Selecting features before splitting: the test rows have already influenced which features exist, so the score that follows is not a measurement of anything.

How bad that gets is not fixed — it depends on the shape of the table. The next figure turns the two available handles, the number of rows and the number of columns, and measures the damage

each one does.



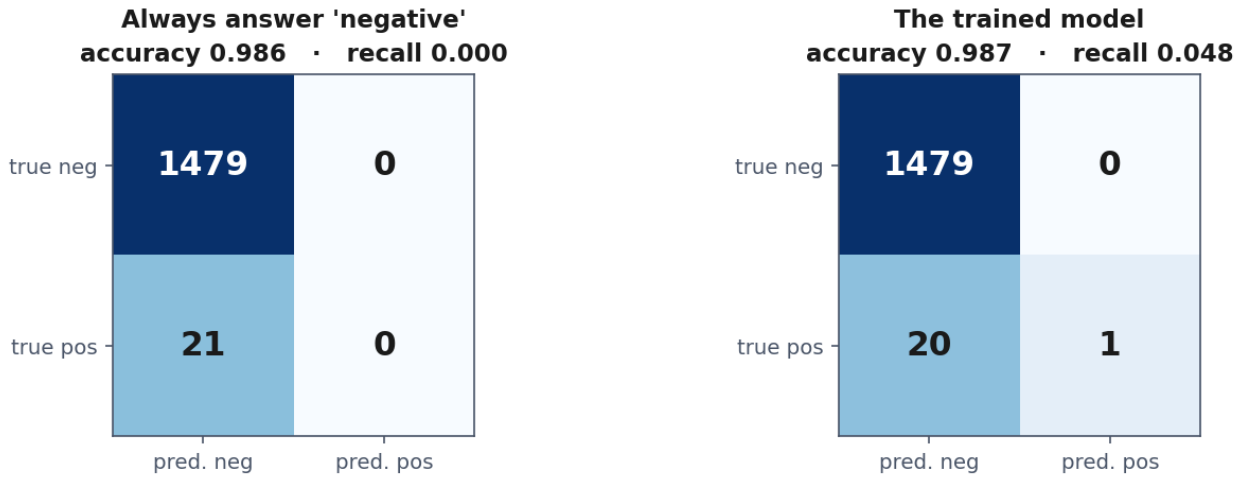
Both panels report the accuracy the leaked pipeline claims, on labels that are coin flips — so the honest answer is the dashed line at 0.500, and every point above it is pure illusion. Each panel turns one handle and holds the other fixed at the value named in its axis label; the two meet at 200 examples and 5000 columns, where both read 0.799. Bars are 95% intervals over 100 repetitions, small enough that the slopes are real.

Read the left panel from right to left, because that is the direction its title describes: starting at 400 examples and halving repeatedly, the claimed accuracy climbs 0.731, 0.799, 0.874, 0.955. Every halving of the data buys roughly eight more points of fiction. The right panel reads the ordinary way and is much flatter: doubling the candidate columns from 5000 to 10000 moves it 0.799 to 0.813, about one and a half points.

That asymmetry is not an accident of this dataset, and it follows from how each side scales. A noise correlation measured on m rows has spread $1/\sqrt{m}$, so halving the examples multiplies it by 1.41. The largest of n such draws grows only like $\sqrt{2\ln n}$, so doubling the columns multiplies it by 1.04. More columns buy more lottery tickets; fewer rows make every ticket pay better.

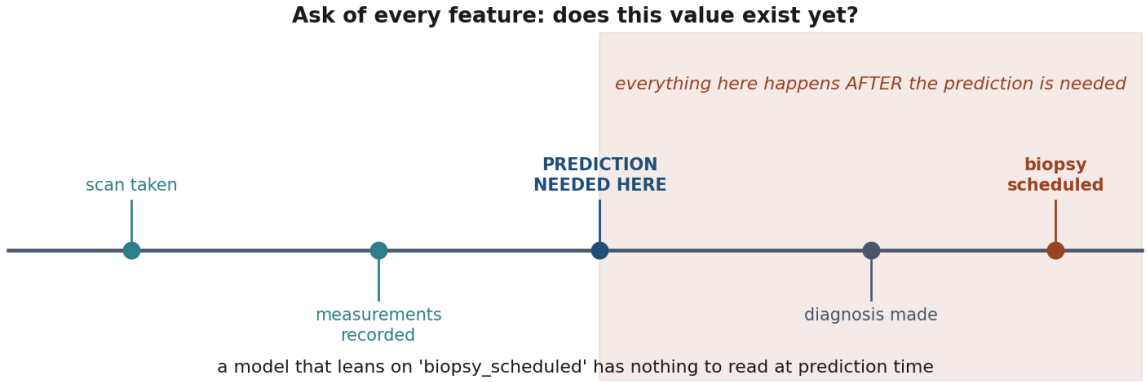
The uncomfortable consequence is the one to remember. Leakage is at its most severe exactly where data is scarce — and scarce data is also where a suspiciously good score is least likely to be questioned, because there was never enough of it to check against anything else.

Imbalanced classes. When 1% of cases are positive, answering "negative" always scores 99%. Accuracy measures the class you are not interested in.



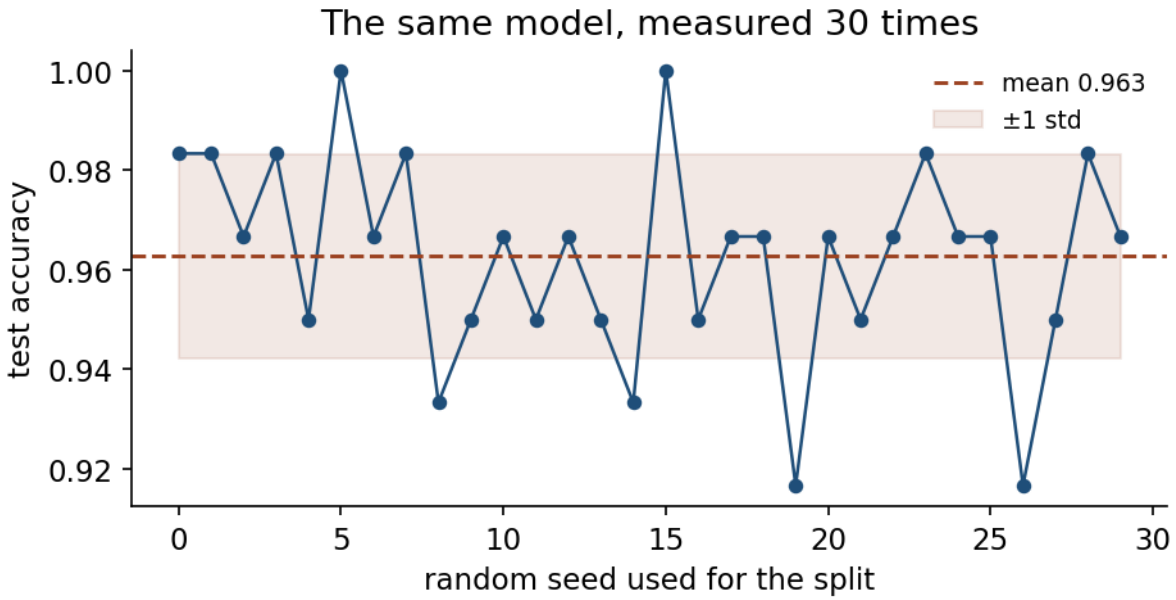
99% accurate and detecting nothing. On a rare-event problem, accuracy measures the majority class and little else.

Shortcut features. A column that is a consequence of the label rather than a predictor of it — a treatment code, a follow-up appointment. The model leans on it and collapses in deployment, where the column does not exist yet. No metric detects this; only knowing how the data was recorded does.



A shortcut feature — something that predicts the label in this dataset for a reason that will not survive contact with new data. Everything to the right of the marked moment happens after a prediction is needed, so none of it can be an input.

Single-split noise. One train/test split gives one draw from a distribution. On a small dataset the spread across splits can exceed the difference between two competing models. A result quoted with no indication of variability is incomplete.



One split is one measurement. Repeat it with a different seed and the number moves — which is why lesson 5 is about measuring properly.

Notebook 03 fits the same model to the same data thirty times, changing nothing but the seed that decides the split. The accuracy runs from **0.917 to 1.000** — a spread of **0.083** around a mean of 0.963. Nothing in that picture is a difference between models, or between hyperparameters, or between anything at all: it is one number, measured thirty times.

That spread is larger than most improvements reported between competing methods, which is the uncomfortable part. Run the code once, as real life allows, and you may report any value in that range while a reader who sees one number cannot tell which you drew. This is not fraud; it is the default behaviour of anyone who runs an experiment once.

The repair is to stop taking one draw. Five-fold cross-validation on this data gives **0.960 ± 0.030** — almost the same centre, now with the spread reported alongside it. Lesson 5 makes that the centre of the course.

---

8. Limits and responsibility

Three things worth stating on day one, since they shape everything that follows.

A model is a compressed summary of its training data, including its injustices. If past decisions discriminated, a model fitted to them will too — and it will do so with the appearance of objectivity, which makes the discrimination harder to challenge than if a person had made it.

Correlation is what these methods find. Nothing in empirical risk minimisation distinguishes a cause from a coincidence that happens to predict well in this sample. A model can be highly accurate and completely wrong about *why*, which is why it can fail abruptly when conditions change.

Accuracy is not the only property that matters. Whether a decision can be explained to the person affected, whether errors are recoverable, and who bears the cost of being wrong are all

engineering requirements, not afterthoughts. They belong in the framing step, not in a paragraph at the end of a report.

---

9. What to do before the next lesson

1. Verify your environment: JupyterLab running, all three notebooks executing top to bottom.
2. Work through the notebooks in order — 01 for the shape of the process, 02 for the taxonomy, 03 for the failure modes.
3. Take the quiz in `Quizzes/`.
4. **Complete the homework** in `Exercises/01_first_workflow.md`, due at the start of Lesson 2.

Notation used in this lesson

Symbol	Meaning
x, X	one input, the design matrix
y, $\hat{y}$	true target, prediction
f	the function the model computes
S, T	the sample used for training, the test set
m, n	number of examples, number of features
L	the loss on a single example
R, $\hat{R}$	the expected risk, the empirical risk
$\mathcal{D}$	the unknown distribution the data is drawn from
p	a measured accuracy, read as a proportion (Section 2.4)
SE	the standard error of that proportion

These carry the same meanings in every lesson of the course. Where a later lesson needs a symbol for something else, it says so in its own table.

Further reading

Resource	Type	Why read it
scikit-learn: common pitfalls	Official docs	The library's own account of leakage and how pipelines prevent it
Turing, *Computing Machinery and Intelligence* (1950)	Paper	The imitation game in Turing's words; shorter and sharper than its reputation
Wolpert & Macready, *No Free Lunch Theorems* (1997)	Paper	Why no single method is best on every problem — the formal statement behind Lesson 6

Resource	Type	Why read it
Sculley et al., *Hidden Technical Debt in ML Systems* (2015)	Paper	What goes wrong once a model leaves the notebook
Géron, *Hands-On Machine Learning*, ch. 1–2	Book	A complementary treatment of the workflow, with a different worked example
`Resources/history_of_ai.md`	Course material	The historical arc in more depth, with primary sources